Practical Assignment

Multi-Agent Hospital Simulation

This assignment accompanies the section **Tools** and focuses on building a simplified, agentic hospital intelligence system. You will implement a multi-agent simulation in which planner and executor agents coordinate diagnostic tasks via a shared context.

The goal is **not** to build a clinically valid system, but to understand how agentic architectures work in high-risk domains, how agents coordinate via shared state, and where limitations and safety concerns arise.

Learning Goals

By completing this assignment, you will:

- Design and implement a multi-agent system in Python
- Model agent roles such as planners and executors
- Use simple ML models as tools inside an agentic workflow
- Work with shared context / memory between agents
- Analyze behaviour, limitations, and risks of agentic systems in medicine

Assignment Overview

You will simulate a hospital diagnostic workflow with:

Planner agents — Decide which diagnostic pathway a patient case should follow.

Executor agents — Run a diagnostic model (e.g. diabetes, heart disease, breast cancer) and report results.

Shared Context (MCP-like layer) — A central data structure used by all agents to read/write tasks and results.

The simulation runs in discrete steps and logs how cases flow through the system.

Environment Setup

1. Create a Virtual Environment

We strongly recommend using a virtual environment, using Python 3.10 or higher.

```
python3 -m venv venv
```

Activate it:

- **Linux / macOS:** `source venv/bin/activate`
- **Windows**: `venv\Scripts\activate`

2. Install Dependencies

```
pip install -r requirements.txt
```

Step-by-Step Tasks

Step 0: Train Diagnostic Models

Use public datasets to train simple classifiers (random forests):

- Diabetes (Pima Indians dataset)
- Breast cancer (sklearn.datasets)
- Heart disease (UCI)

```
python 0_train_models.py
```

Step 1: Understand the Baseline

The baseline code is provided in `1_basic_mcp.py`. Read and execute it carefully, paying attention to:

Shared Context (MCP)

A shared context object that stores incoming patient cases, assigned diagnostic tasks, and diagnostic results. All agents interact only through this context, not directly with each other.

Planner Agent

Reads unassigned cases, decides which diagnostic domain applies, and creates tasks in the shared context.

Executor Agent

Reads tasks assigned to its specialty, runs the corresponding ML model, produces predictions, and writes results back to the context.

Step 2: Run the Baseline Simulation

Execute the baseline and observe what happens. Think carefully about its design and evaluation.

Step 3: Replace the Rule-Based Planners with an LLM Agent

To make the simulation more meaningful and realistic, replace the 3 rule-based planner agents with 1 LLM planner agent. The skeleton code is provided in `2_llm_mcp.py`.

Notice the `make_case_text` function (which you are **not allowed to modify**): it creates a combined textual representation of a patient's features drawn from all 3 datasets.

Your tasks:

1. Implement `call_llm_fn()` using a real LLM (Ollama, OpenAI, or Gemini — a Google account gives limited free Gemini access).
2. Implement the `step()` method of `LLMPlannerAgent`. The agent must decide which ML models to run (one or more) and which disease is the primary concern per case.
3. Write analysis code to produce evaluation tables and plots of the LLM planner's triage performance.

Report Requirements

Submit a **brief written report** (PDF, max 2 pages) that answers the following questions. **You do not need to write a narrative essay** — clearly numbered answers are fine. You will elaborate on your reasoning during the oral examination.

Questions to Answer in the Report

Section A — Baseline Architecture

Q1. Draw or describe (in at most 5 sentences) the flow of a single patient case through the baseline system.

Q2. In the baseline, each case is sampled from a dataset and assigned to exactly one disease domain. What does this imply about how the planner makes its decision? Is this realistic as a triage mechanism?

Q3. The models in `0_train_models.py` are trained and evaluated on the same data used to generate simulation cases. Why is this a problem? What would a fair evaluation look like?

Section B — LLM Planner Design

Q4. Which LLM did you use, and how did you call it (API, local, etc.)? Briefly justify your choice.

Q5. The `make_case_text` function presents a patient as a mix of features from three separate datasets. Why is this medically unrealistic, and does that matter for the purposes of this assignment?

Q6. Describe one concrete design choice you made in your `step()` implementation (e.g., how you handle parsing failures, abstentions, multi-route cases) and explain why you made it.

Section C — Results

Q7. Report the triage precision and recall for each disease. Include the confusion matrices for the ML executor models. What do these numbers tell you about how well the system works?

Q8. The LLM planner can route a case to multiple diseases simultaneously. Give one example scenario (real or hypothetical) where this is beneficial, and one where it could be harmful.

Section D — Reflection

Q9. Identify one safety risk specific to using an LLM as a planner in a medical triage context. How could it be mitigated?

Q10. Is it fair to evaluate triage performance by comparing LLM routing decisions against ground-truth labels from the original datasets? Explain any reservations you have.

Oral Examination

Each student will have a short oral examination (~10–15 minutes) with the instructor. You will be asked to:

- Walk through your code and explain what specific parts do
- Answer follow-up questions on your report answers
- Reason about hypothetical modifications or failure scenarios

You are expected to understand and be able to explain every line of code you submit. The oral examination is designed to verify this.

Optional Extensions (Bonus)

You may optionally:

- Add another disease from the UCI repository
- Add an evaluator or verifier agent
- Simulate human-in-the-loop decision making
- Add uncertainty thresholds and abstention logic
- Introduce retrieval from external knowledge sources

These are not required but can improve your grade if done well and explained clearly in the oral.

Submission

Submit a **zip file** containing:

- All code files with a `README` explaining how to run them
- Trained model files (or clear instructions to reproduce them)
- The brief report as a PDF

Make sure the code runs from a clean unzip following the instructions above.

Grading Overview

Component	Weight
Written report (Q1–Q10)	60%
Oral examination	40%

Important Notes

- This assignment is educational, not clinical.
- Do not claim medical validity.
- Clear structure, reproducibility, and genuine understanding matter more than raw performance.
- AI tools may be used as a coding aid, but you must be able to explain every part of your submission.

Good luck — and treat your agents responsibly.